Autonomous Agents and Multiagent Systems

A series of notes on the course AAaMS as taught in Politecnico di Milano by Francesco Amigoni and someone else

Autonomous Agents and Multiagent Systems
Negotiation Part 2 - 11/10
- Rubinstein's Alternating Offer Protocol
- Monotonic Concession Protocol
- Zeuthen Strategy

Computational Social Choice - 18/10
- Social Welfare Function
  - Arrow's Theorem
- Social Choice Function
  - Voting Rule

Negotiation Part 2 - 11/10

We now focus on the agreement found by the agent, not on the policy.

We'll define a protocol, see how the agents behave in it, and the study the possible outcome that will be reached by the agent.

Let's start with the first protocol:

Rubinstein's Alternating Offer Protocol

I make an offer, the other agent makes a counter offer, I make a new offer, and so on, until we find some agreement.

We assume that we have 2 agents, a, b.

We assume there is one issue over which the agents are negotiating. It can be though as an object that can be divided by the 2 agents.
Let's think of a cake. It can be divided by the 2 agents. We'll call the piece of cake that goes to the agent a x^a. x^b is the part of the cake that goes to b.

we assume that $x^a + x^b = 1$ (it forms the complete cake).

The set of outcomes is trivially the set of all the pairs (x^a, x^b) such that $x^a + x^b = 1$.

in math: $O = (x^a, x^b) | x^a + x^b = 1$ where O stands the set of possible Outcomes.

We assume that time is evolving in discrete period: $t = 1, 2, \ldots$.

We assume that players play in turn, with player a playing first:
So agents a makes offers in odd timestamps, while b makes offers in even timestamps. This is the core of "Alternating" Offer protocol.

the ones above are all and only the assumptions that the agents must do.

Let's talk about the utility of a: it depends on how big the cake is and on the timestamp in which the offer is agreed.

$$U^a(x^a, t) = x^a \cdot \delta_a^{t-1}$$

where δ_a is the discount factor and $0 \leq \delta_a \leq 1$

$\delta = 1 \rightarrow$ the agent is very patient.

$\delta = 0 \rightarrow$ the agent must accept the first offer otherwise it gets zero.

and obviously:

$$U^b(x^b, t) = x^b \cdot \delta_b^{t-1}$$

Rubinstein shows that it is possible to compute a subgame perfect equilibrium:

$$x^a = \frac{1 - \delta_b}{1 - \delta_a \delta_b}$$

$$x^b = \frac{\delta_b}{1 - \delta_a \delta_b}$$

It means that the agent will not find convenient to deviate from these values.

we notice that if

$$\delta_a = 0; \delta_b = 1 \rightarrow x_a = 0;\ x_b = 1$$

$$\delta_a = 1; \delta_b = 0 \rightarrow x_a = 1;\ x_b = 0$$

$$\delta_a = 0.5; \delta_b = 0.5 \rightarrow x_a = \frac{2}{3};\ x_b = \frac{1}{3}$$

in the last case, the agent a gets more cake because it started first.

The cases above have been computed considering that the negotiation can go on forever, but what happens if we have a deadline $t = n$? (n is the last timestamp in which an offer can be done).

Let's suppose even that $\delta_a = \delta_b = \delta$

$n = 1 \rightarrow$ agent b will just accept, can't do anything else. So agent a will propose $offer(1, 0)$ more precisely:

$$\text{STRAT-A}(1) = \text{OFFER}(1, 0)$$
$$\text{STRAT-B}(1) = ACCEPT$$

$n = 2$

$$\text{STRAT-A}(1) = \text{OFFER}(1 - \delta, \delta)$$
$$\text{STRAT-A}(1) = \text{ACCEPT}$$

which translates into:

$$U^a(1 - \delta, 1) = 1 - \delta$$
$$U^b(\delta, 1) = \delta$$

let's revise what we have just written:
The agent a prefers to offer to b δ part of the cake because the utility of b can't be more than δ (in fact $U^b(1, 2) = \delta$), because this way a gets at least $1 - \delta$ as utility, so it's a win win.

let's generalize:

deadline n, $\delta_a = \delta_b = \delta$

$$\text{STRAT-A}(n) = \begin{cases} \text{if a's turn: OFFER(1,0)} \\ \text{if b's turn: ACCEPT} \end{cases}$$

$$U^a(n) = \begin{cases} \delta^{n-1} & \text{if it's a's turn} \\ 0 & \text{if it's b's turn} \end{cases}$$

$t < n$

$$\text{STRAT-A}(t) = \begin{cases} \text{if a's turn: OFFER}(1 - \delta x^b(t+1), \delta x^b(t+1)) \\ \text{if b's turn: if } U^a(x^a, t) \geq UA(t+1) \text{ then ACCEPT else REJECT} \end{cases}$$

Monotonic Concession Protocol

- agents a, b ; U^a, U^b
- O
- simultaneous offer protocol
- $t = 1, 2, \ldots$

From the point of view of agent a:

1. $x^{(a)} \leftarrow \arg\max_{x \in O} U^a(x)$

2. propose $x^{(a)}$

3. receive $x^{(b)}$

4. if $U^a(x^{(b)} \geq U^a(x^{(a)}))$ then ACCEPT $x^{(b)}$ and RETURN
   else $x^{(a)} \leftarrow x \in O$ such that $U^b(x) \geq U^b(x^{(a)})$ and $U^{(a)} \geq 0$

5. goto 2

This algorithm can be applied only when you know the utility of the other agent, which does not happen very often in the real world...

Moreover, the converge of this algorithm can b very long.

Another problem: It can happen that agent a accepts the offer of agent b and agent a accepts the offer of agent b, but they are not the same! we have two different agreements: we need to introduce some tie-breaking to decide the agreement. we can pickup just one at random , or we can find the middle point of the offer and come up with a single agreement.

There is a variant of this protocol that is called the Zeuthen Strategy

Zeuthen Strategy

This is the point of view of a

1. $x^{(a)} \leftarrow \arg\max_{x \in O} U^a(x)$

2. propose $x^{(a)}$

3. receive $x^{(b)}$

4. if $U^a(x^{(b)} \geq U^a(x^{(a)}))$ then ACCEPT $x^{(b)}$ and RETURN

5. $\text{risk}_a \leftarrow \frac{U^a(x^{(a)})-U^a(x^{(b)})}{U^a(x^{(a)})}; \text{risk}_b \leftarrow \frac{U^b(x^{(b)})-U^b(x^{(a)})}{U^b(x^{(b)})}$

6. $\text{if risk}_a < \text{risk}_b \text{ then } x^{(a)} \leftarrow x \in O \text{ such that risk}_a > \text{risk}_b$

7. $\text{goto } 2$

Property: the final agreement found via this algorithm is a pareto optimal agreement, which means that there is no another agreement that is not worse for one agent and strictly better for the other one.

it finds an agreements which is a Nash BARGAININ solution. It is an agreement that maximizes the product of the two utilities.

Computational Social Choice - 18/10

Applications:

- Aggregation of preferences.
- Formation of coalitions.
- Sometimes you have multiple robots that have to keep a formation, for example military patterns.
  It is based on the fact that there is a leader that is moving and all the other robot are keeping a fixed distance and angle from him. They should also avoid to collide with each other.
  What happens if the leader fails? In this case there is a kind of voting for choosing the new leader, and this is the social choice.

Let's start from the model that we will adopt for studying computational social choice.

we'll have:

- a number of ***agents*** $N = \{1, 2..., n\}$
- a set of **alternatives**, also called **candidates** (in the case of political elections for example) U. we assume $|U| \geq 2$.
- ***preferences***, denoted by the symbol *preference relation* $>_{\sim i}$ (it is an ordering over the alternatives).
  for it being a proper ordering we need to constraint such relations in the following way. we need to say that such relation is
  - complete: $\forall a, b \in U : a >_{\sim i} b \text{ or } b >_{\sim i} a$ (the or is not generally exclusive)
  - transitive: $\forall a, b, c \in U : \text{if } a >_{\sim i} b \text{ and } b >_{\sim i} c \text{ then } a >_{\sim i} c$

Observations: agents and candidates could be disjoint or not depending on the case. In political election agents are people and politicians, while candidates are politicians.

Now let's introduce some symbols:

- ***preference profile*** $R = (>_{\sim 1}, >_{\sim 2}, >_{\sim n})$ (it is an array of preferences, one for each agent).
  a preference can be structured as a pair of agent, candidate. So a preference profile can be represented as a list of pairs.
- a ***set*** of all possible ***preference relations over*** $U : \mathscr{R}(U)$
  $>_{\sim i} \in \mathscr{R}(U)$

Mathematicians game:
We are at a party in which people can drink only one beverage.
6 mathematicians want to drink, in preference order, milk,wine,beer.
5 mathematicians want to drink, in preference order, beer,wine,milk.
4 mathematicians want to drink, in preference order, wine, beer,milk.
So:

- we have 15 agents: $n = 15$.
- we have 3 alternatives: $|U| = 3$
- for the mathematician number 1, who belongs to the 6 mathematicians cited first, we'll have $\text{milk} >_{\sim 1} \text{wine} >_{\sim 1} \text{beer}$. and so on for all the mathematicians.

It is not easy to aggregate together different preference relations with a global choice that is satisfactory for all people. It depends even on the mechanism of voting.

Social Welfare Function

It is a function that given a preference profile returns a preference relation.

$f : \mathscr{R}(U)^n \to \mathscr{R}(U)$

$f(R) \; = \; >_\sim$

This is what the function should do, there are different ways to implement it.

We want this function to have the following properties

- <u>Pareto Optimality</u>
  Informal definition: assume that every agent thinks that candidate a is better than b. then it is reasonable to assume that a will be better than b even in the global relation.
  Formal definition:
  $\text{if } a >_i b \text{ then } a > b$
  the one above has been written assuming the following: $a >_{\sim i} b \text{ but not } b >_{\sim i} a$, which is not the general case.
  the consequence assumes instead that: $a >_\sim b \text{ but not } b >_\sim a$.
  I had to write the formal definition this way because the actual formal definition would be complex to be written, so Amigoni preferred specifying the assumptions afterwards.
- <u>Independence of irrelevant alternatives (IIA)</u>
  imagine this starting situation
  $$f \begin{cases} \ldots >_{\sim 1} a >_{\sim 1} \ldots >_{\sim 1} b >_{\sim 1} \ldots \\ \ldots >_{\sim 2} a >_{\sim 2} \ldots >_{\sim 2} b >_{\sim 2} \ldots \\ \ldots >_{\sim 3} a >_{\sim 3} \ldots >_{\sim 3} b >_{\sim 3} \ldots \end{cases} \to \ldots >_\sim a >_\sim \ldots >_\sim b >_\sim \ldots$$
  In a nutshell we want to say that if you swap the red and yellow elements, and, as we can see, a and b have the same ordering over all the agents, and in case a and b have anyways the same order in the global relation, we can say that our formulation has IIA.

  In case of IIA the ordering of a and b in the global relation does not dependent on how it is called the yellow element.
  Formally:
  $\text{if } R|\{a,b\} = R'|\{a,b\} \text{ then } >_\sim |\{a,b\} =>'_\sim |\{a,b\} \text{ where } f(R) =>_\sim \text{ and } f(R') =>'_\sim$
- <u>Non-Dictatorship</u>
  A good social welfare function is such that there is no such an agent which imposes the ordering over the other agents. This is Non-Dictatorship.
  Formally:
  $\text{there is no } i \in N \text{ such that } f(R) =>_{\sim i} \forall >_{\sim i}$

Arrow's Theorem

There is no any social welfare function that satisfies the 3 properties above when you have at least 3 candidates.

The property that is mostly dropped is the IIA.

Social Choice Function

The Social Welfare Function can be too powerful in many situations. take for example the problem of electing 3 new members in parliament over 10 possible candidates. you just need to know the 3 most voted ones, you don't care about the others.

So now we'll define the social choice function

$f : \mathscr{R}(U)^n \times \mathscr{F}(U) \to \mathscr{F}(U)$

where $\mathscr{F}$ is the powerset of U: $\mathscr{F} = 2^U$, which is the set of all feasible sets of alternatives.

$A \in \mathscr{F}(U)\ A \subseteq U$

$f(R, A) = A' \qquad A' \subseteq A$

Voting Rule

one special case of the social choice function is the ***voting rule***.

Given a preference profile it returns a set of candidates.

$f : \mathscr{R}(U)^n \to \mathscr{F}(U)$
$f(R) \to A'$

f is resolute when $|f(R)| = 1$